

Lernaufgabe

LE 06 – Klassenentwurf

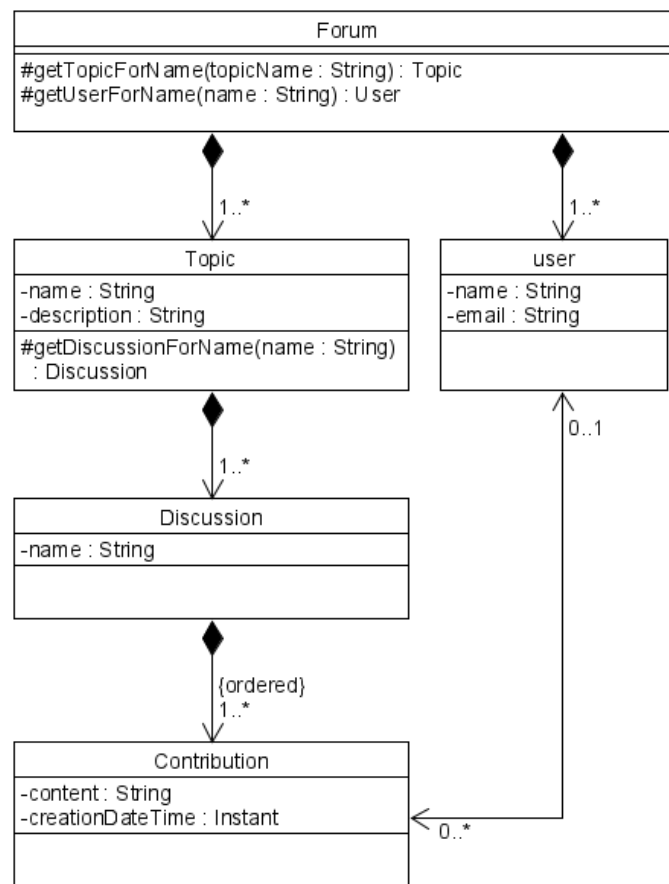
Lernziel

Sie in der Lage, die GRASP Prinzipien und Pattern konsequent für den Klassenentwurf anzuwenden, Designs damit zu evaluieren sowie Designs mit einem Design-Klassendiagramm und Interaktionsdiagrammen zu modellieren bzw. darzustellen.

Einleitung

In dieser Lernaufgabe werden ausgehend von einem ersten Design-Klassendiagramm (DCD) für ein Diskussionsforum (kurz Forum) verschiedene Systemoperationen aus Use-Case-Szenarien entworfen und implementiert.

Alle Aufgaben basieren auf dem folgenden Design-Klassendiagramm (DCD), das aus dem Domänenmodell für ein Forum abgeleitet wurde.



Das DCD stellt ein unvollständiges Design eines API (Application Programming Interface) zur Verwaltung eines Forums dar. Ein Forum besteht aus einem oder mehreren Themen,



und jedes Thema besteht aus einem oder mehreren Diskussionen. Eine Diskussion besteht aus einem oder mehreren Beiträgen.

Wichtige Use Cases eines Forums sind:

- UC01 – Sign In
- UC02 – Sign Out
- UC03 – Read a Discussion
- UC04 – Post a Contribution
- UC05 – Add a New Discussion
- UC06 – Add a new Topic
- UC07 – Show Statistics

Jeder Benutzer kann im Forum die Beiträge lesen. Um irgendetwas verändern oder hinzufügen zu können, muss sich ein Benutzer authentifizieren.

Für das Lösungsdesign sind Anforderungen, dass die Domänenschicht strikt entkoppelt ist von der UI-Schicht und der Art der Authentifizierung, da verschiedene Frontends (Web, JavaFX) und Authentifizierungssysteme damit verwendet werden können.

Aufgabe 6.1: Controller

In einem ersten Schritt wird der Controller für die Realisierung der Systemoperationen für die verschiedenen Use-Case-Szenarien festgelegt.

Aufgabe

Bestimmen Sie den Controller für das Forum. Entscheiden Sie sich auch, ob Sie einen Fassaden- oder Use-Case-Controller verwenden.

Vorgehen

1. Analysieren Sie die Problemstellung und bestimmen Sie den Controller.
2. Diskutieren Sie, ob ein Fassaden- oder Use-Case-Controller eingesetzt werden soll. Was ist die Begründung für die eine oder andere Variante?

Hinweise, Tipps

- Keine.

Ergebnis

Die Klasse, die als Controller fungiert mit einer kurzen Begründung für den Entscheid.

Zeit: 5'



Aufgabe 6.2: Systemoperation «getNbrOfContributions()»

Die Systemoperation «getNbrOfContributions()» wird im Hauptszenario des Use Case «UC07 – Show Statistics» verwendet und gibt die Anzahl Diskussionsbeiträge («Klasse Contribution») im gesamten Forum zurück.

Aufgabe

Erstellen Sie ein Kommunikationsdiagramm, das das Design der Systemoperation darstellt und ergänzen Sie das Design-Klassendiagramm. Erklären Sie Ihre Design-Entscheide kurz, indem Sie auf die GRASP Prinzipien und Pattern verweisen.

Vorgehen

1. Analysieren Sie die Problemstellung und modellieren Sie die Systemoperation mit einem Kommunikationsdiagramm.
2. Evaluieren Sie Ihr Design, indem Sie die Zuordnung von Verantwortlichkeiten und Kollaborationen mittels GRASP begründen und annotieren Sie die Objekte und Operationen entsprechend.
3. Ergänzen Sie das DCD mit den neuen und geänderten Attributen und Operationen.

Hinweise, Tipps

- Sie brauchen die Zugriffe auf Container Objekte (die Objekte, die die 1..* Beziehung realisieren) nicht im Detail zu modellieren. Wenn Sie dies so machen, sollten Sie aber Methoden einfügen, die diese Zugriffe abkapseln oder Sie vermerken über Notizen, wo Sie auf die Container Objekte zugreifen.

Ergebnis

Ein aktualisiertes Design-Klassendiagramm und ein Kommunikationsdiagramm.

Zeit: 15'

Lernaufgabe

LE 06 – Klassenentwurf

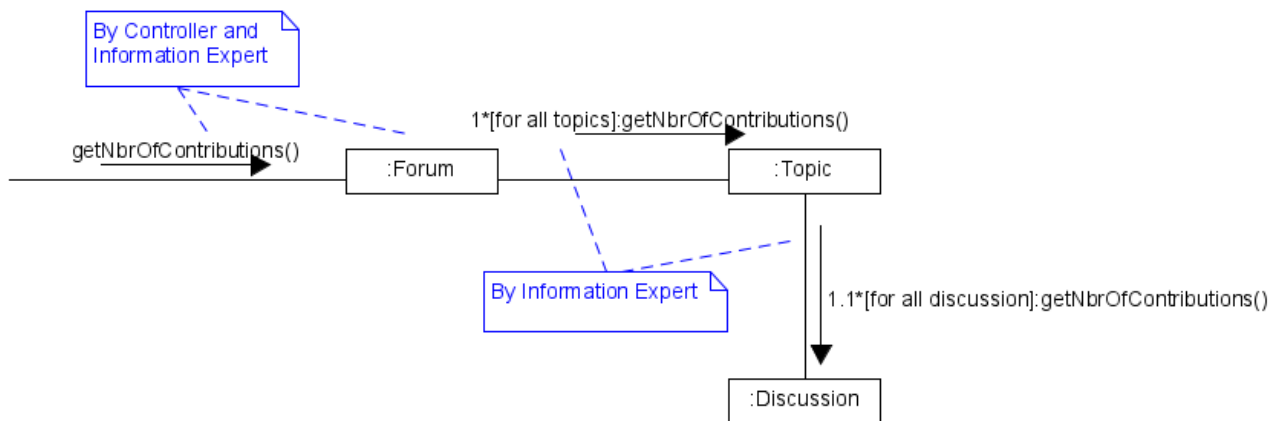
Musterlösung

Aufgabe 6.1: Controller

- **Forum** (als «Facade Controller»)

Da es im Moment nur wenige Systemoperationen sind und der Controller jeweils nur Verarbeitung delegiert, ist ein Fassaden- oder Root-Controller gut genug.

Aufgabe 6.2: Systemoperation «getNbrOfContributions»



Die Systemoperation zur Berechnung der Anzahl Beiträge («Contributions») wird so realisiert, dass über alle Stufen der Zugehörigkeitshierarchie («containment hierarchy») iteriert wird und die jeweiligen Zwischenergebnisse aufsummiert werden.

Alle oben aufgeführten Methoden müssen zusätzlich im DCD eingetragen werden. Beachten Sie die hierarchische Nummerierung. Bezüglich Low Coupling wird auf die Diskussion von `getTotal()` der Fallstudie verwiesen.

Das Addieren der Zwischenergebnisse findet in den jeweiligen «`getNumberOfContributions()`» Methoden statt und wird hier gar nicht modelliert.

Das Design direkt über User alle Contributions zu ermitteln, liefert eine falsche Anzahl, wenn Benutzer gelöscht werden!



Aufgabe 6.3: Systemoperation «addNewDiscussion(...)»

Die Systemoperation «addNewDiscussion(accessToken : byte[], userName : String, topicName : String, discussionName : String)» im Hauptszenario des Use Case «UC05 – Add New Discussion» fügt eine neue Diskussion einem bestehendem Topic hinzu. Es muss überprüft werden, ob so eine Diskussion bereits existiert und in diesem Fall muss eine Exception geworfen werden.

Als Vorbedingung für den Use Case, muss sich der Benutzer eingeloggt haben (Use Case «UC01 - Sign In»). Die Authentifizierung erfolgt über einen Single-sign-on-Mechanismus und liefert nach erfolgreichem einloggen ein Access-Token zurück, das den Benutzer berechtigt, Veränderungen im Forum vorzunehmen. Dieses Access-Token muss bei jedem Aufruf geprüft werden (Validität und Gültigkeit).

Aufgabe

Erstellen Sie ein Sequenzdiagramm, das das Design der Systemoperation zeigt und ergänzen Sie das Design-Klassendiagramm. Erklären Sie Ihre Design-Entscheidung kurz, indem Sie auf die GRASP Prinzipien und Pattern verweisen.

Vorgehen

1. Analysieren Sie die Problemstellung und modellieren Sie die Systemoperation mit einem Sequenzdiagramm.
2. Evaluieren Sie Ihr Design, indem Sie die Zuordnung von Verantwortlichkeiten und Kollaborationen mittels GRASP begründen und annotieren Sie die Objekte und Operationen entsprechend.
3. Ergänzen Sie das DCD mit den neuen und geänderten Attributen und Operationen.

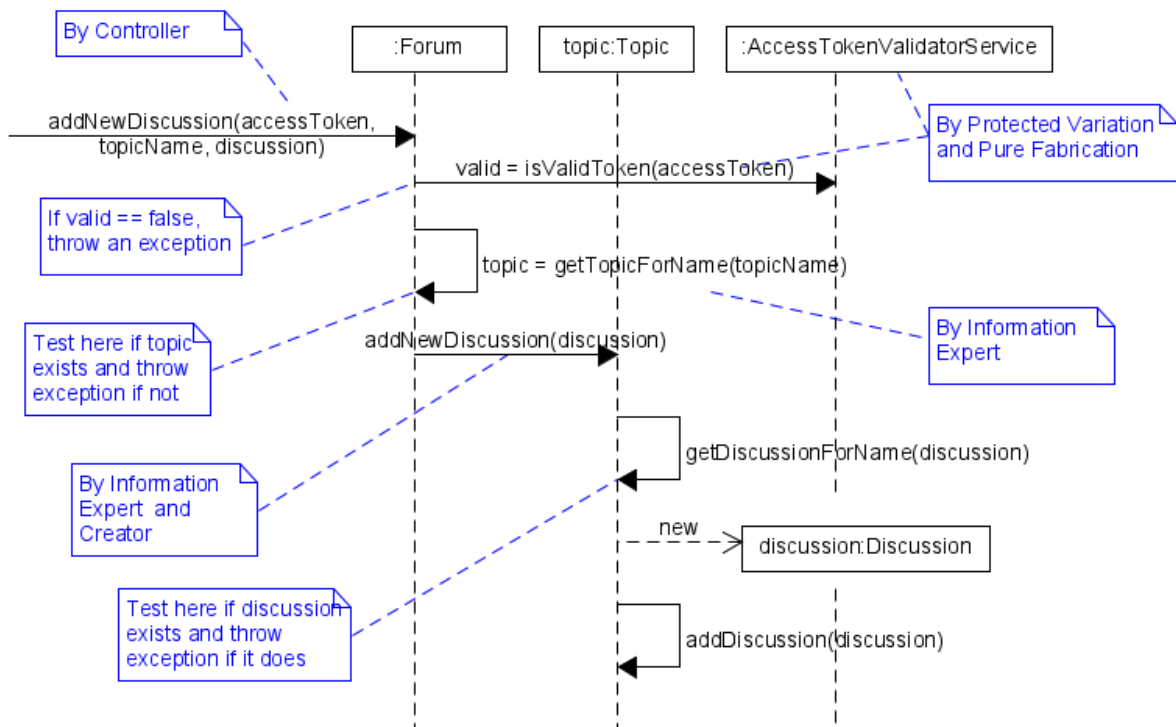
Hinweise, Tipps

- Sie brauchen die Zugriffe auf Container Objekte (die Objekte, die die 1..* Beziehung realisieren) nicht im Detail zu modellieren. Wenn Sie dies so machen, sollten Sie aber Methoden einfügen, die diese Zugriffe abkapseln oder Sie vermerken über Notizen, wo Sie auf die Container Objekte zugreifen.

Ergebnis

Ein aktualisiertes Design-Klassendiagramm (DCD) und ein Kommunikationsdiagramm.

Zeit: 30'

Aufgabe 6.3: Systemoperation «addNewDiscussion()»

Vom Fassaden-Controller aus wird zuerst das Access-Token validiert. Dann wird das richtige Thema («Topic») gewählt. Auf dem Topic wird die Diskussion («Discussion») erzeugt. Die Diskussion muss dann noch mit dem Topic verknüpft werden.

Die Authentifizierung ist gemäss Aufgabenstellung ein Variationspunkt. Darum wird für die Validierung des Access-Tokens ein Interface «AccessTokenValidatorService» mit der Methode «isValidToken()» erstellt (analog für die Authentifizierung gemäss I in SOLID Interface Segregation). Zur Laufzeit kann dann eine konkrete Instanz eines AccessTokenValidator-Service dem Forum injiziert werden (Konstruktor).

Das Sequenzdiagramm zeigt im Wesentlichen nur den Erfolgsfall. Die Fehlerbehandlung ist in den Kommentaren angegeben. Ihre Modellierung mittels UML würde das Diagramm nicht nur aufblasen, sondern auch sehr unleserlich machen.

Alle oben aufgeführten Methoden müssen zusätzlich im DCD eingetragen werden (s. DCD auf der nächsten Seite). Die Zugriffe auf die Container Klassen wurden nicht explizit modelliert, dafür aber entsprechende Methoden auf der jeweiligen Domänenklasse angelegt (getXYZForName(...)). Diese können auch dafür verwendet werden, eine Exception zu werfen, wenn das Domänenobjekt mit dem gesuchten Namen nicht vorhanden ist. Werden diese Tests explizit eingezeichnet, kann dies leicht zu einem unübersichtlichen Diagramm führen.

Die Implementation folgt exakt den Diagrammen. Für die Erzeugung der aktuellen «Instant» wurde auf die abstrakte Klasse «java.time.Clock» zurückgegriffen. Man kann eine Variante davon erzeugen, die wirklich die aktuelle Systemzeit mit der eingestellten Zeitzone repräsentiert. Es gibt aber auch eine Variante, die immer dieselbe Zeit liefert. Diese ist primär für

Testfälle gedacht. Die Erzeugung der aktuellen «Instant» wird der Controller Klasse zugeordnet, da dadurch nur eine Instanz von Clock notwendig ist, die bei der Instanziierung dem Controller «Forum» übergeben wird.

Aktualisiertes DCD für beide Systemoperationen

